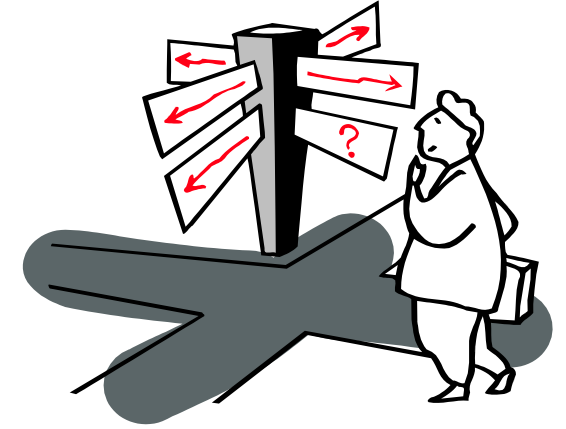


CHAPTER 3 – Software Architecture



- Introduction
 - + When, Why and What?
 - + Functional vs. Non-functional

- *Quality Attributes*

- + Coupling and Cohesion
- + Patterns
- Macro architecture
 - + Layered Architecture
 - + Pipes and Filters
 - + Blackboard Architecture
 - + Model-View-Controller
- Micro Architecture
 - + Observer
 - + Abstract Factory
 - + Adapter (a.k.a. Wrapper)
- Other Patterns
 - + Microservices

- Architecture in UML
- Architecture Assessment
 - + ATAM
- Agile Architecture
 - + Spike
- + *Guiding Principles*
- + *Architecture Decision Register*
- + *Y-Statement*
- + *GuardRails*
- *Generative AI and Architecture*
- Conclusion
 - + Correctness & Traceability

Literature (1/2)

Software Engineering Text Books

- [Somm05]: chapter "Architectural Design"
- [Pres00]: chapter "Architectural Design"

Books on Software Architecture

- [Shaw96] Software architecture: perspectives on an emerging discipline, Mary Shaw, David Garlan, Prentice-Hall, 1996.
 - + The book introducing software architecture.
- [Bass22] Software architecture in practice (4th edition), Len Bass, Paul Clements, Rick Kazman, Addison-Wesley, 2022.
 - + A very deep and practical treatment of software architecture, *incl. ATAM abd Attribute-Driven Design*. (The book received an award.)
- [Erd21] *Continuous Architecture in Practice*, Murat Erden, Eoin Woods, Addison-Wesley, 2021
 - + *How to do Software Architecture in an Agile Context*

Articles

- [Kruc95] Philippe Kruchten "The 4+1 View Model of Architecture ", IEEE Software, November 1995 (Vol. 12, No. 6) pp. 42-50.
 - + A paper that illustrates convincingly the need for various perspectives on the design of a system.

Literature (2/2)

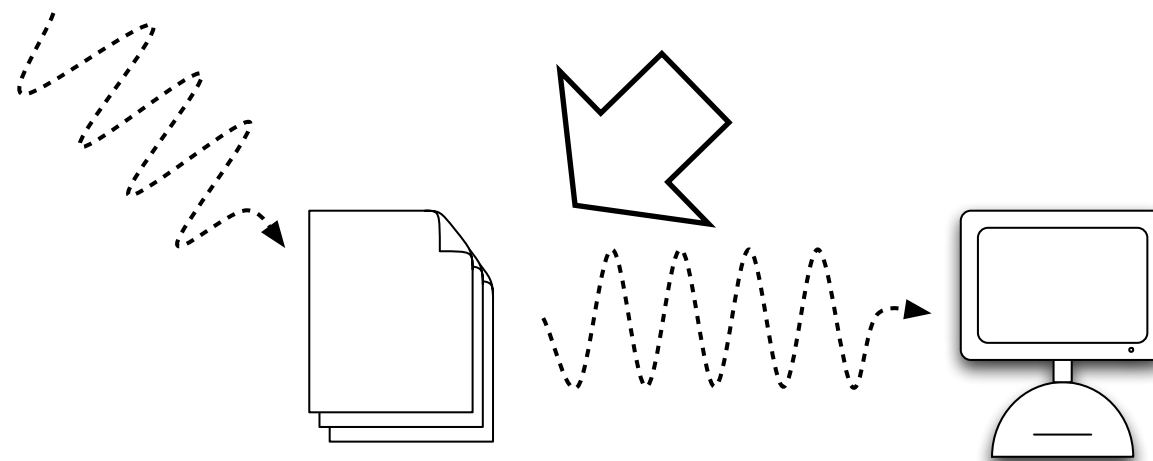
Pattern Language

- [Foot97] Big Ball of Mud, Brian Foote, Joseph Yoder; Fourth Conference on Patterns Languages of Programs (PLoP '97/EuroPLoP '97)
+ <http://www.laputan.org/mud/mud.html>; most popular architecture.

Pattern Catalogues

- [Busc98] Pattern-Oriented Software Architecture: A System of Patterns, Frank Buschman, Regine Meunier, Hans Rohnert, Peter Somerlad, Michael Stal, Wiley and Sons, 1996.
+ Introduces architectural styles in pattern form. Also covers some design patterns and idioms.
 - > At architecture (= "macro-architecture") level
- [Gamm95] Design Patterns: Elements of Reusable Object-Oriented Software, Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides, Addison-Wesley, 1995.
+ The classic; commonly referred to as the "Gang of Four (GOF)"
 - > At design (= "micro-architecture") level

When Architecture?



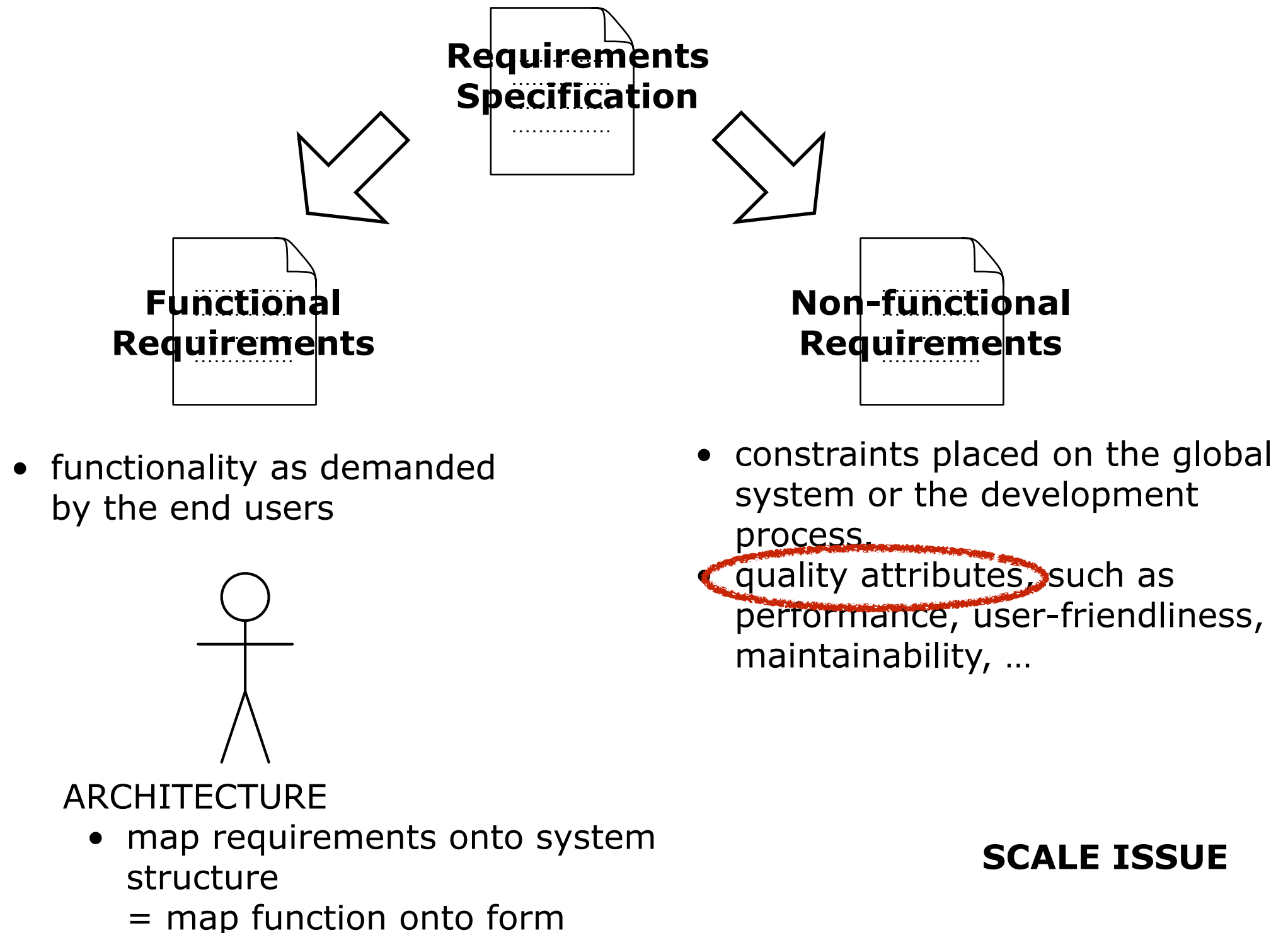
Designing a software system requires *course-grained decomposition*
⇒ organize work in the development team

Conway's law

Organizations which design systems are constrained to produce designs which are copies of the communications structure of these organizations. [Conw68]

- If you have 4 groups working on a compiler; you'll get a 4-pass compiler

Why Architecture



Characteristics of a Gothic Cathedral



Public Domain

Q

Architecture as a Metaphor

Parallels

- Architects are the technical interface between the customer and the contractor.
- A poor architectural design cannot be rescued by good construction technology.
- There are architectural styles or schools.
 - + (e.g., “ghotic” in buildings;
“client-server” in software)

Differences

- Buildings are tangible, software is intangible.
 - > Software Architecture is often expressed via metaphors.
- Buildings are rather static, software is quite flexible.
 - > The underlying architecture allows to anticipate changes.
- Building architecture is supposed to be aesthetic.
 - > Buildings avoid to mix styles; in software heterogeneity is considered good.
- A building architect carries legal responsibilities.
 - > Usually a building architect is not employed by the constructor.



 Public Domain

What is Software Architecture?

Software Architecture

- A description of *components* and the *connectors* between them.
 - + Typically specified in different views to show the relevant functional and non-functional properties.

Component

- An encapsulated part of a software system with a designated *interface*.
 - + Components may be represented as modules (packages), classes, objects or a set of related functions. A component may also be a *subsystem*.

Subsystem

- A component that is a system in its own right, i.e. can operate independently

Connector (a.k.a. Relationships)

- A connection between components.
 - + There are *static* connectors that appear directly in source code (e.g., use or import keywords) and *dynamic* connectors that deal with temporal connections (e.g., method invocations).

View

- Represents a partial aspect of a software architecture that shows specific *functional and non-functional properties*.

Functional vs. Non-functional Properties

- See [Bush98]

Functional property

- Deals with a particular aspect of the system's functionality. Usually in direct relationship with a particular use case or conceptual class.

Non-functional property

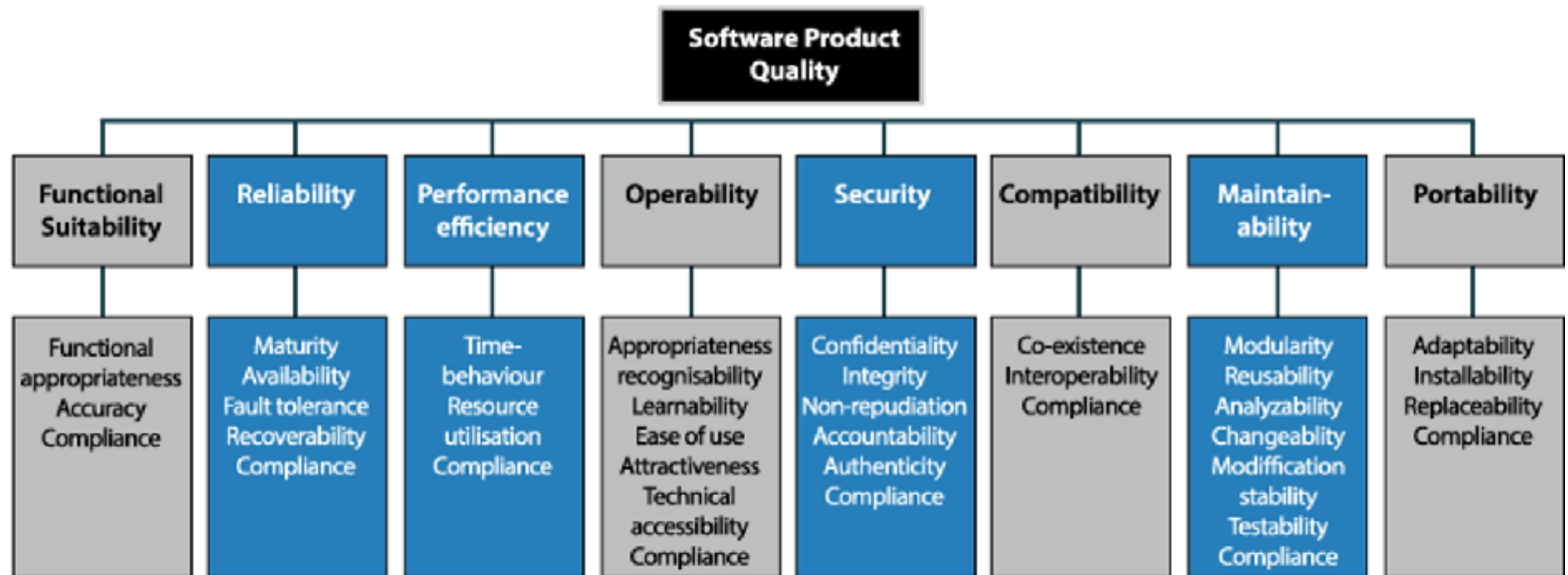
- Denotes a constraint placed on the global system or the development process. Typically deals with quality attributes that cross-cut the whole system design and are quite intangible.
- Typical non-functional properties
 - + Changeability; systems must evolve or perish
 - + Interoperability; interaction with other systems
 - + Efficiency; use of resources such as computing time, memory, ...
 - + Reliability; system will continue to function even in unexpected situations
 - + Testability; feasibility to verify that requirements are covered
 - + Reusability; ability to reuse parts of software system or process for constructing other systems

Architecture is about tradeoffs

ISO 25010 Software Quality

****New****

<https://iso25000.com/en/iso-25000-standards/iso-25010>



Quality Attributes Exercise

****New****



You are part of an architecture team for an MRI scanner. Rank the following quality attributes from most to least important.

- Cost
- Reliability
- Maintainability
- Security
- Ease of use



Coupling and Cohesion

Coupling

- Measure of strength for a connector (i.e., how strongly is a component connected with other components via this connector)

Cohesion

- Measure of how well the parts of a component belong together (i.e., how much does the functioning of one part rely on the functioning of the other parts)

- > Coupling and cohesion are criteria that help us to evaluate architecture tradeoffs.
- > Minimize coupling and maximize cohesion

However ...

- The perfect trade-off corresponds to a component that does nothing!
- Coupling at one level becomes cohesion at the next.
 - > More qualitative trade-off analysis is necessary

Patterns

Pattern

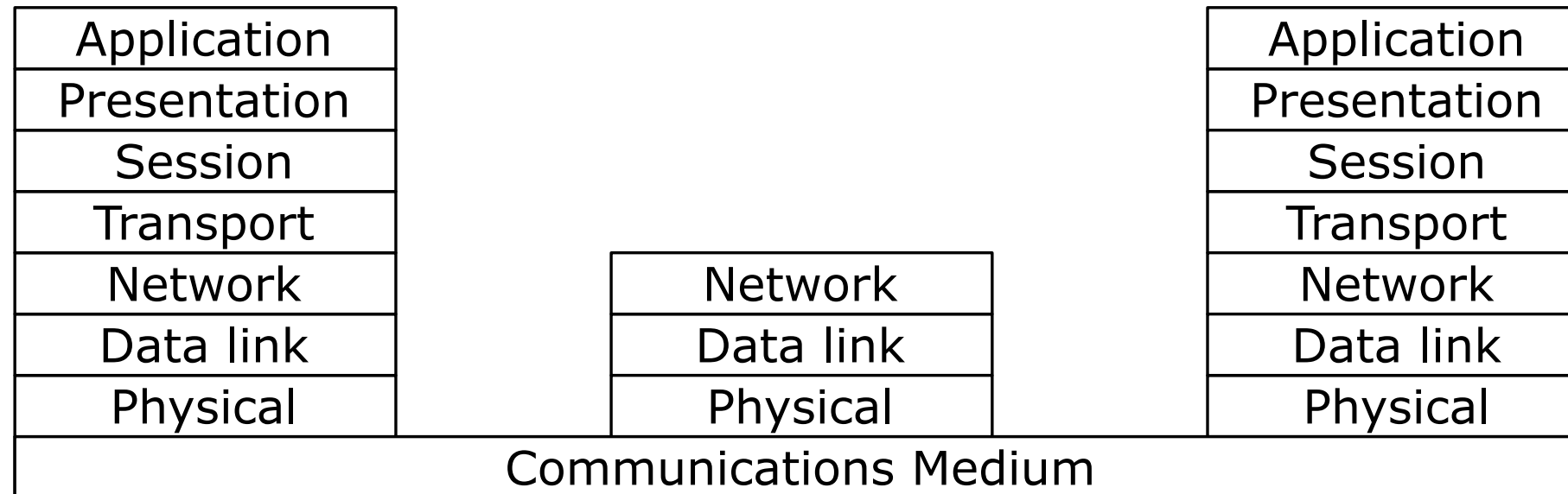
- The essence of a *solution* to a *recurring problem* in a particular context.
 - + Experts recall a similar solved problem and *customize* the solution.
 - + Patterns document *existing* experience.
 - + The context of a pattern states *when (and when not)* to apply the solution.
 - + A pattern lists the *tradeoffs* (a.k.a. forces) involved in applying the solution.

Pattern Form

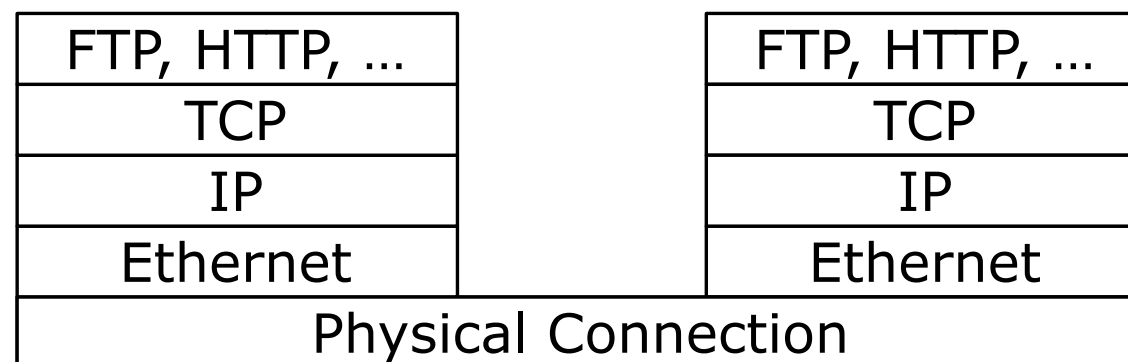
- Patterns are usually written down following a semi-structured template.
 - + Patterns always have a *name*
 - + Patterns allow experts to have deep design discussions in a few words!

Layered Architecture in Networks

OSI Reference Model



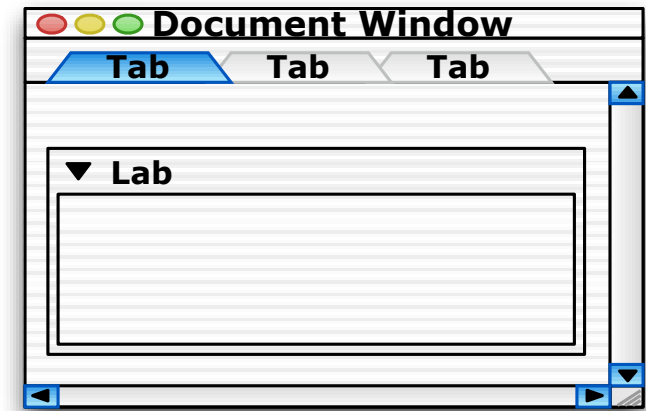
TCP/IP Stack



3-Tiered Architecture

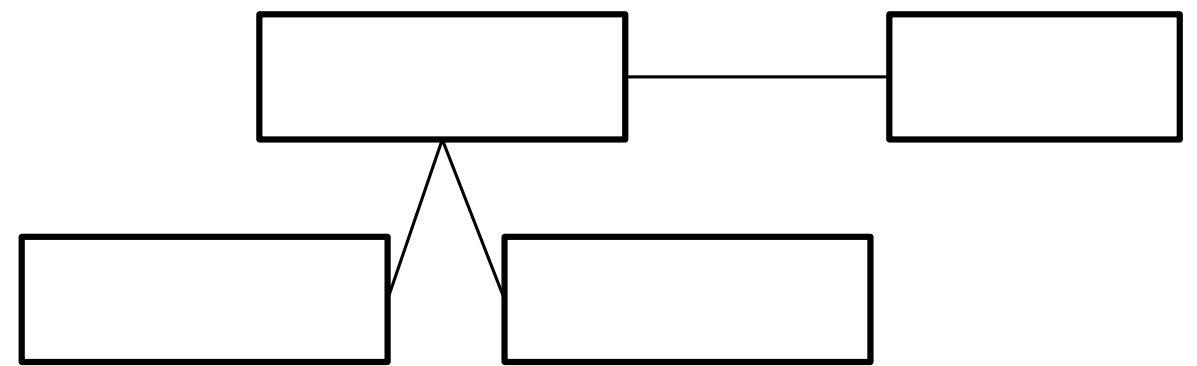
Application Layer

- Models the UI and application logic



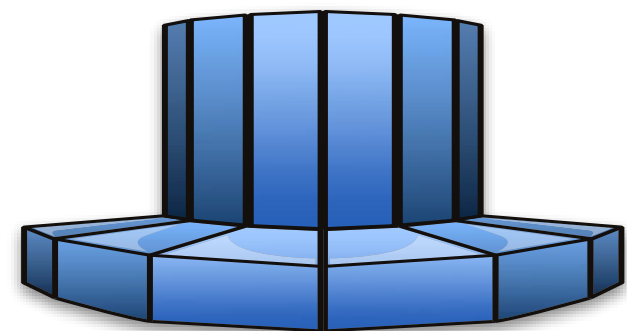
Domain Layer

- Models the problem domain (usually a set of classes)



Database Layer

- Provides data according to a certain database paradigm (usually relational database)



Pattern: Layered Architecture

Context

- Requirements imply various levels of abstraction (low & high level)

Problem

- Need for portability and interoperability between abstraction levels

Solution

- Decompose system into layers;
each layer encapsulates issues at same level
- Layer n provides services to layer $n + 1$
- Layer n can only access services at layer $n - 1$
 - + Call-backs may be used to communicate back to higher layers
 - + Relaxed variant allows access to all lower layers

Tradeoffs

- How stable and precise can you make the interfaces for the layers?
- How independent are the teams developing the different layers?
- How often do you exchange components in one layer?
- How much performance overhead can you afford when crossing layers?

Pipes and Filters Examples

UNIX shells

- `tar cf - . | gzip -cfbest | rsh hcross dd`

data source =
current directory

pipe

filter =
compress

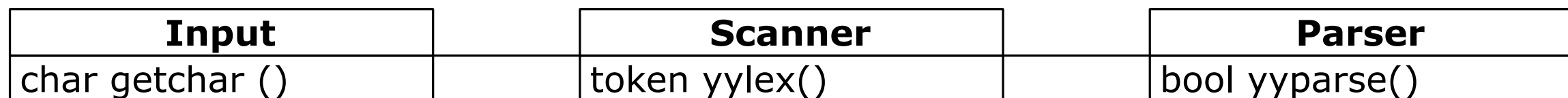
pipe

data sink =
remote host

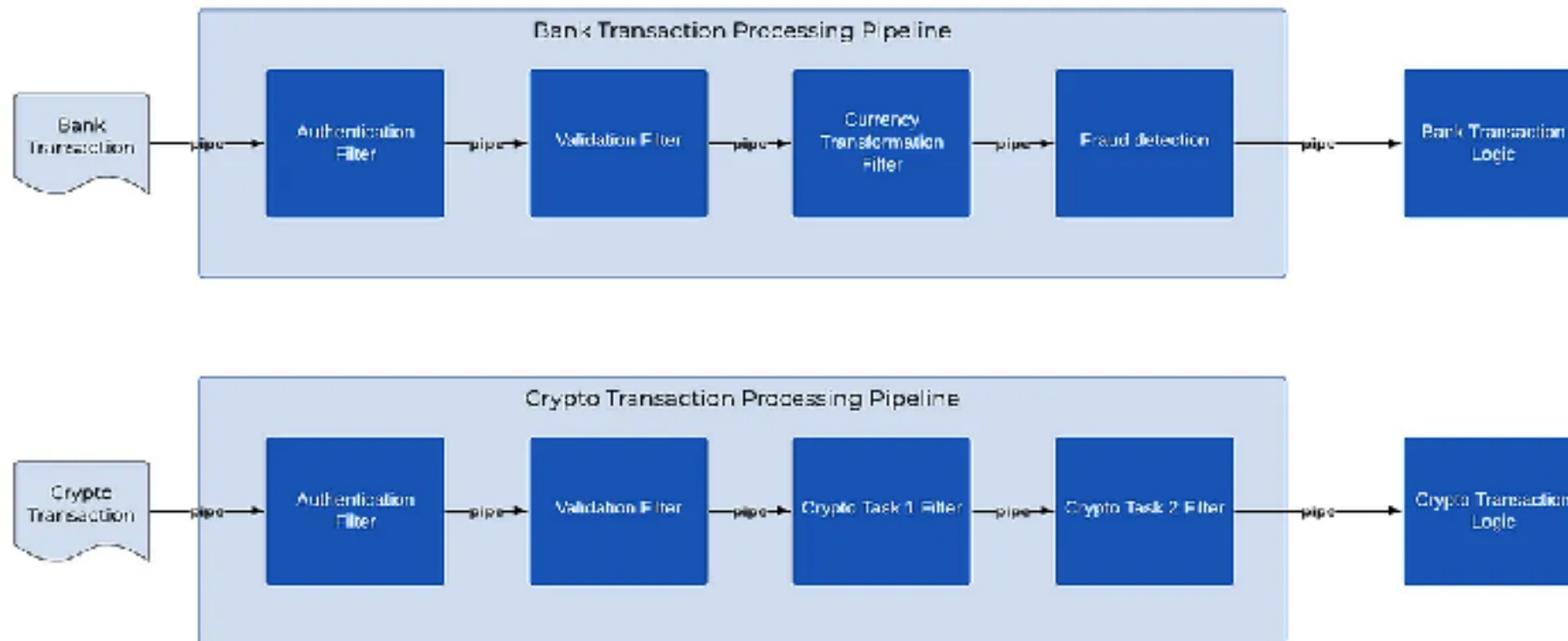
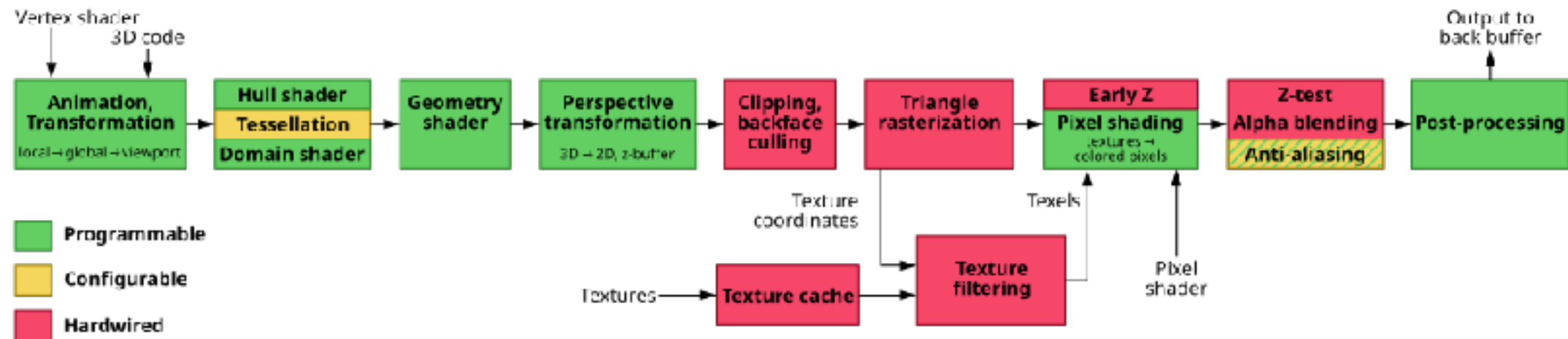
Many CGI-scripts for WWW-forms

- data source is some filled in web-form
 - filters are written via a number of scripting languages (perl, python)
 - data sink is generated web page
- + Example: wiki-web pages (<http://c2.com/cgi/wiki>)

Scanners & Parsers in Compilers



Other Pipelines



Pattern: Pipes and Filters

Context

- Processing data streams

Problem

- Flexibility (and parallelism) is required

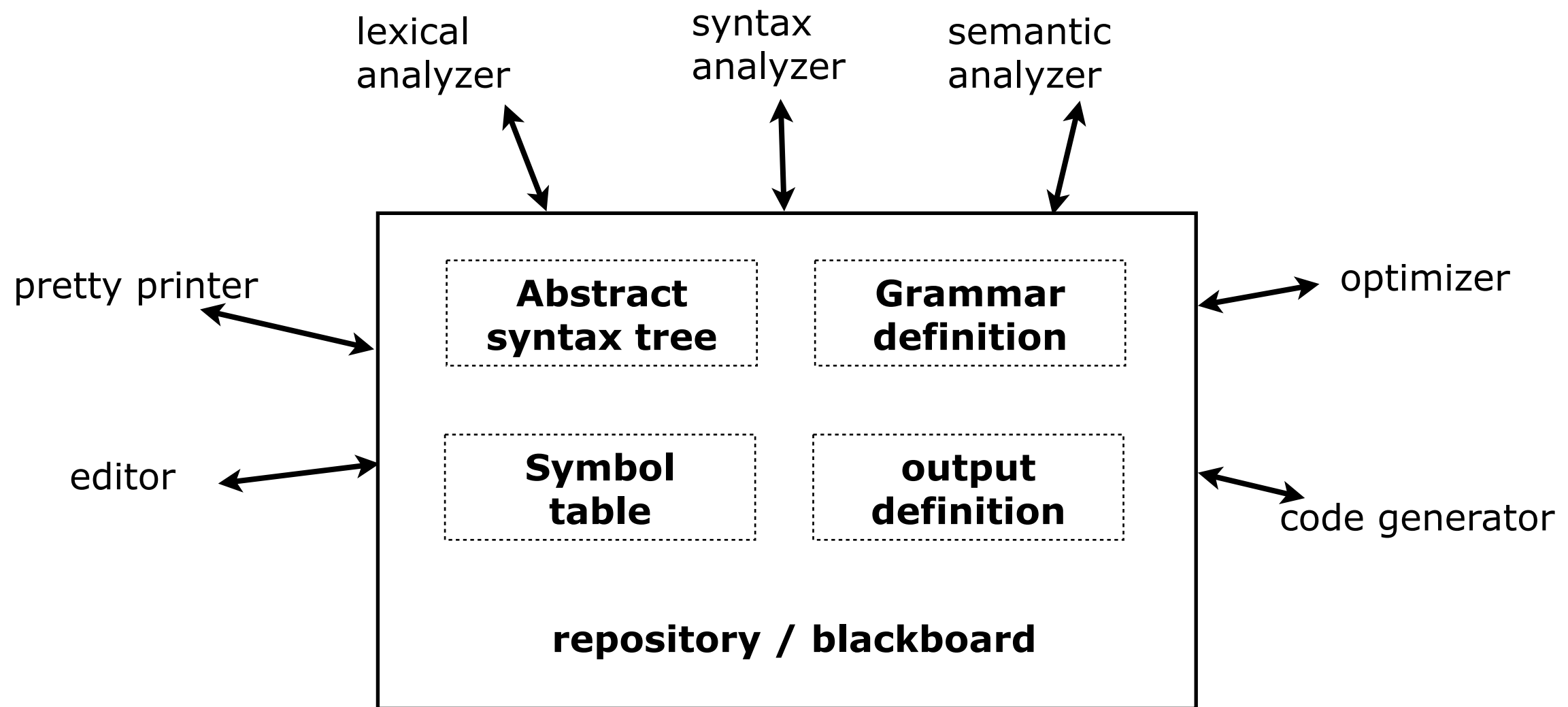
Solution

- Decompose system into filters, each with 1 input- and 1 output stream
- Connect output from one filter to input of another
 - > Need a data source and data sink
- Variants
 - + Push filter: filter triggers *next* one by pushing data on the output
 - + Pull filter: filter triggers *previous* one by pulling data from the input

Tradeoffs

- How often do you change the data processing?
- How well can you decompose data processing into independent filters?
 - + Sharing data other than in/out streams must be avoided
- How much overhead (task switching, data transformation) can you afford?
- How much error-handling is required?

Compilers as Blackboard Architecture



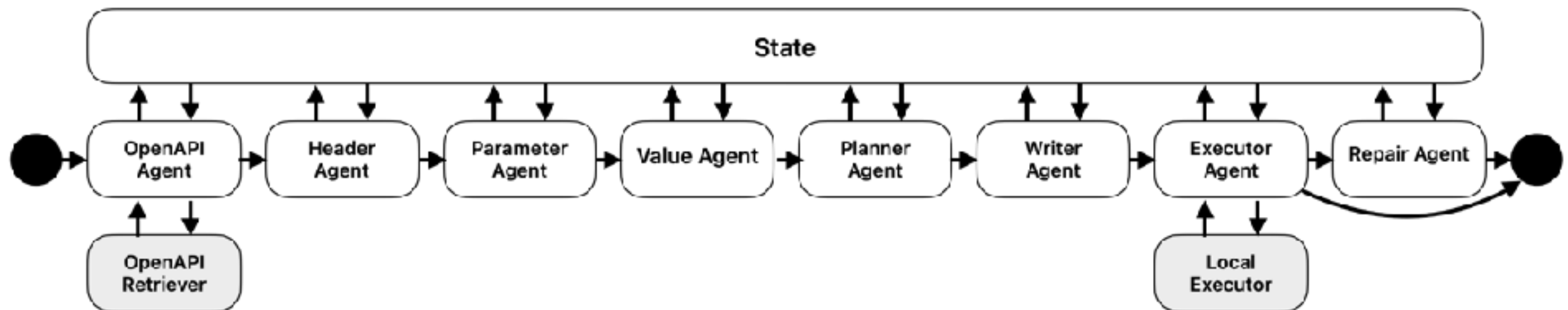
Agentic Workflow as Blackboard Architecture

****New****

Break the task up in multiple steps and assign each task to a specialised agent.

Agents exchange information via a centralised data source.

Control is sequential.



Pattern: Blackboard (a.k.a. Repository)

Context

- Open problem domain with various partial solutions

Problem

- Flexible integration of partial solutions

Solution

- Decompose system in 1 blackboard, several knowledge sources and 1 control
 - + *Blackboard* is common data structure
 - + *Knowledge sources* independently fill and modify the blackboard contents
 - + *Control* monitors changes and launches next knowledge sources

Tradeoffs

- How well can you specify the common data structure?
- How many partial solutions exist? How will this evolve?
- How well can you compose an overall solution from the partial solutions?
- Can you afford partial solutions that do not contribute the current task?

Quizz



Why is a repository better suited for an integrated development environment than pipes and filters?



LLVM

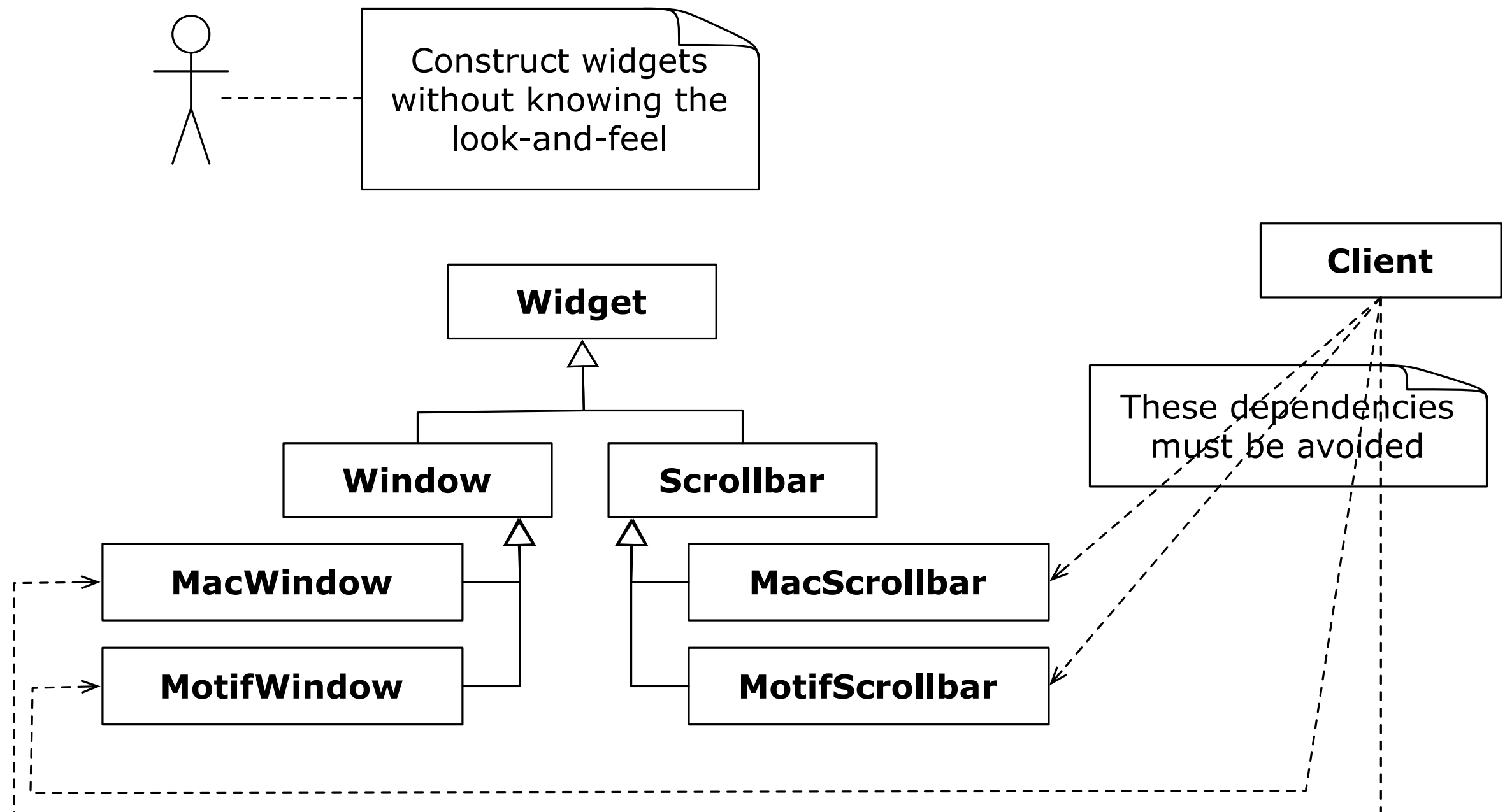


CLION

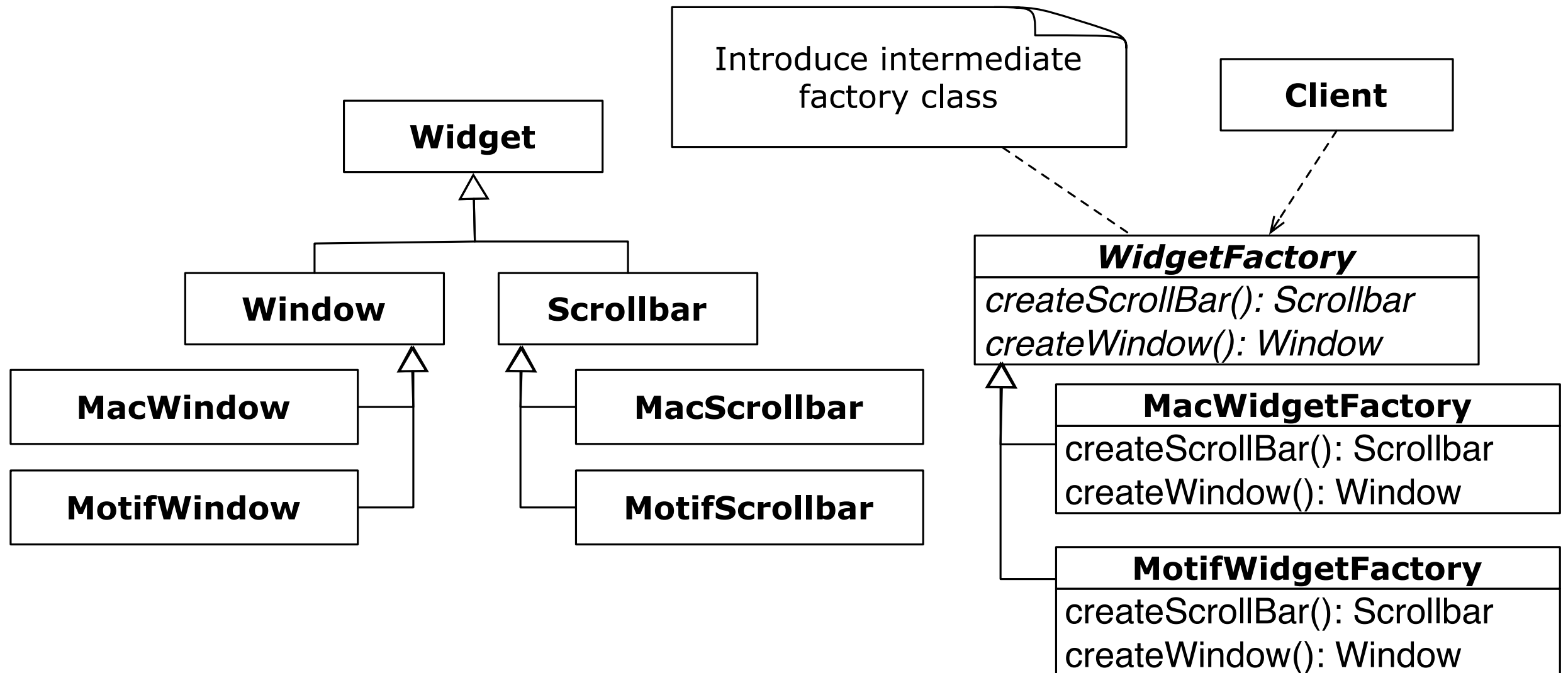


VisualStudio

Problem: Constructor Dependencies



Solution: Abstract Factory



Pattern: Abstract Factory

Context

- Class hierarchy with abstract roots representing a family of objects
+ concrete leaves representing particular configurations

Problem

- Invoking constructors implies tight coupling with concrete leaves instead of abstract roots

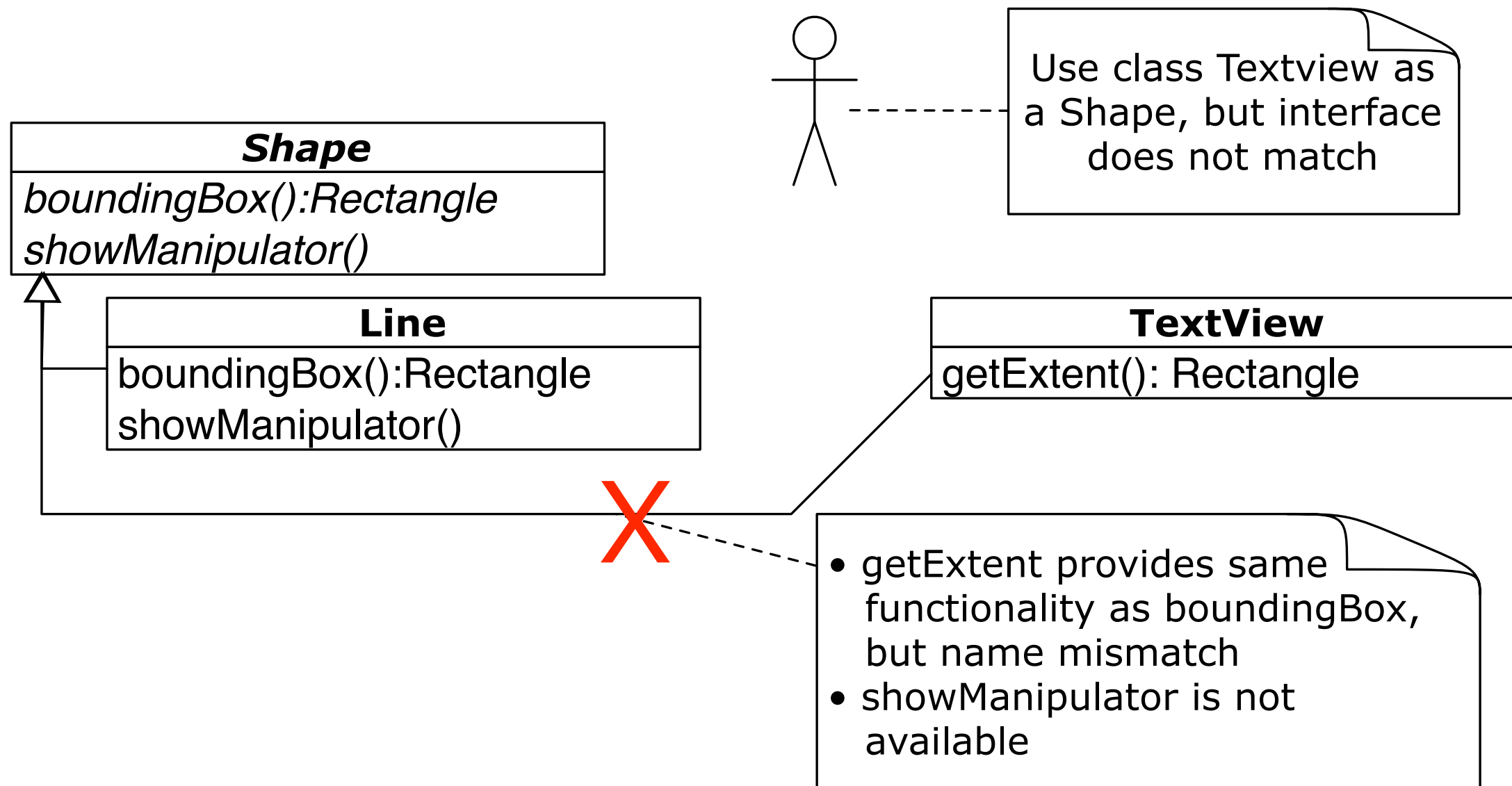
Solution

- Create an abstract factory class with operations for creating all abstract roots
- Create concrete factory classes for all possible configurations.

Tradeoffs

- How many members in the family? How many configurations?
- When do you switch configurations?
- How strict are the configurations?
- Can clients rely on the abstract interfaces?

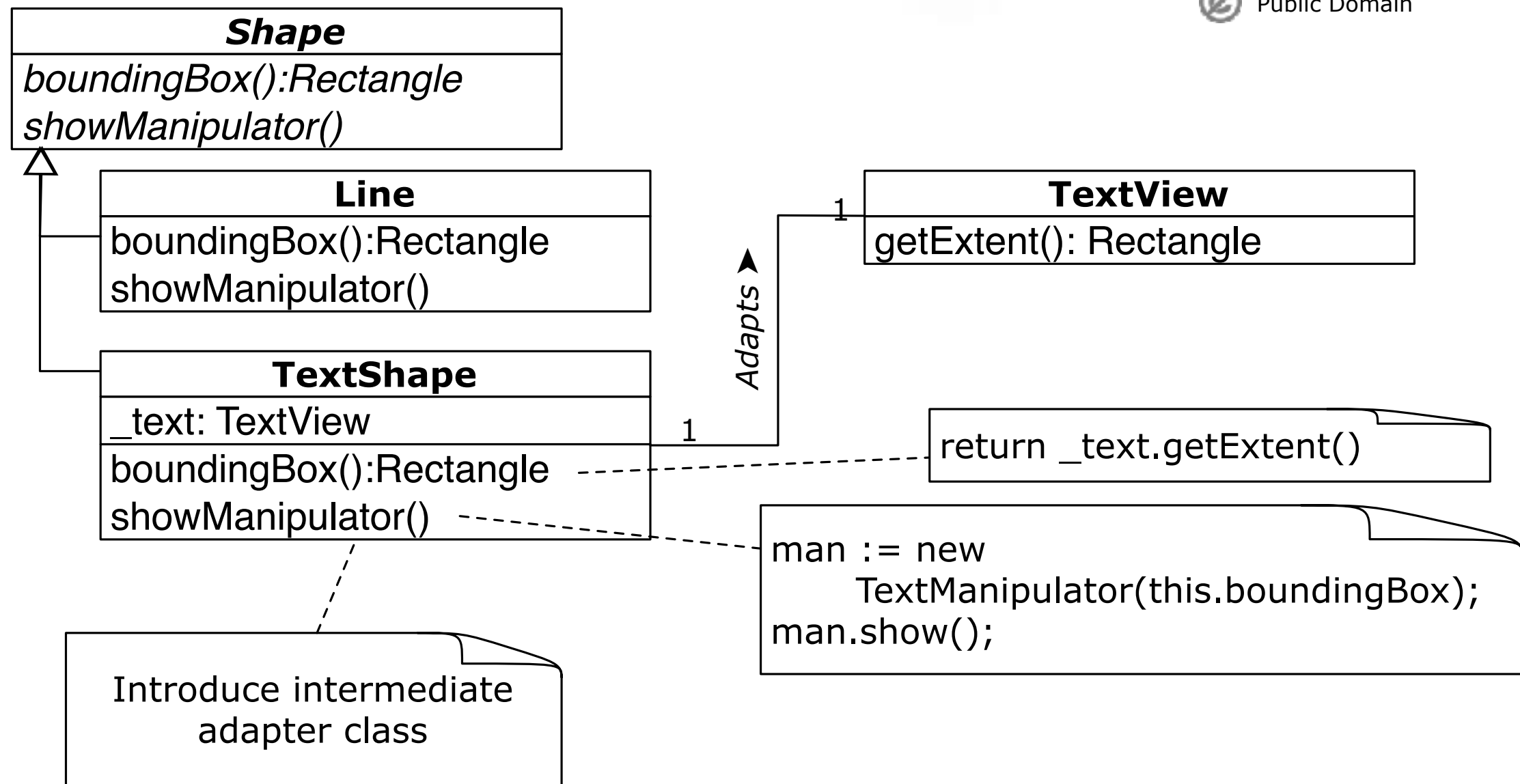
Problem: Interface Mismatch



Solution: Adapter



Public Domain



Pattern: Adapter (a.k.a. Wrapper)

Context

- Merge two separately developed class hierarchies

Problem

- Class provides (most of) needed functionality but interface does not match

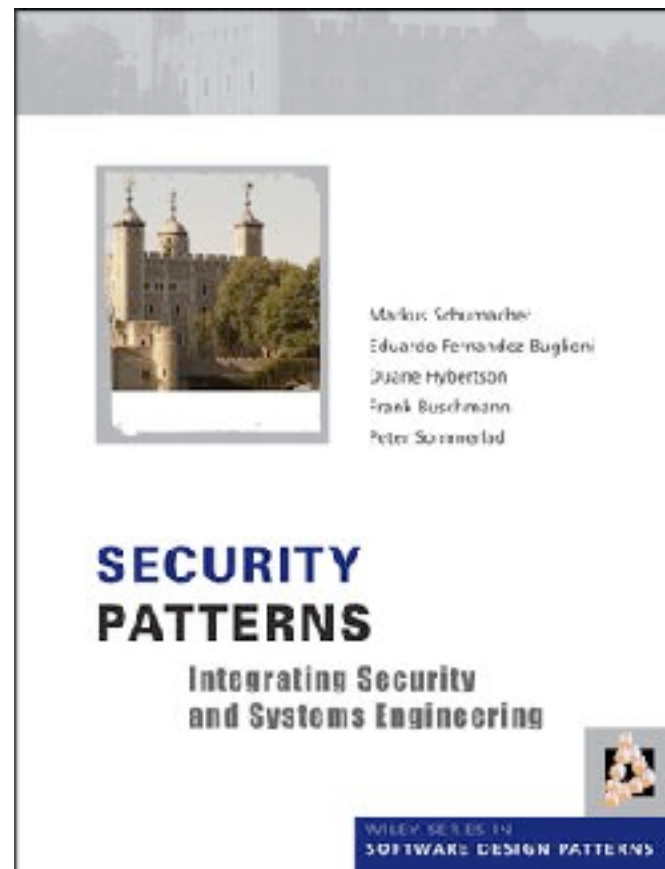
Solution

- Create an adapter class with one attribute of adaptee class
- Adapter class translates required interface into adaptee class

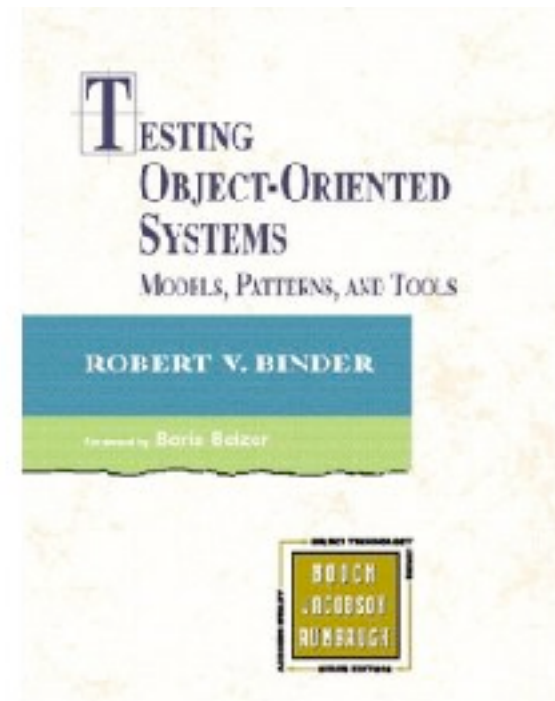
Tradeoffs

- How much adapting is required?
 - + For one class
 - + For the whole hierarchy
- How will the separately developed classes evolve?
- Does the merging work in one direction or in both directions?
- How much overhead in performance and maintenance can you afford?

Other Pattern Catalogues

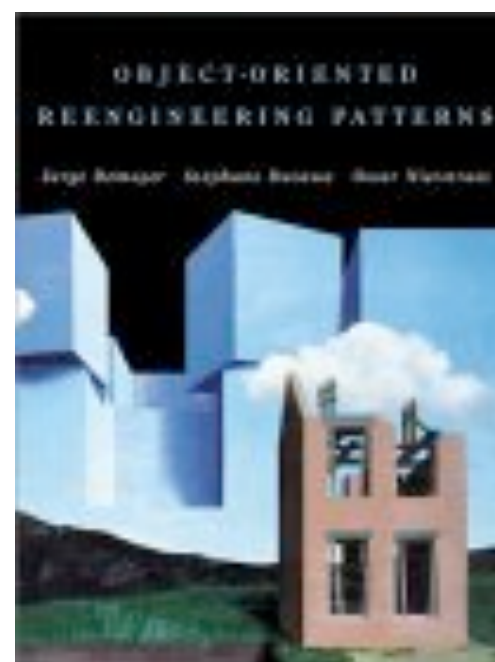


Security

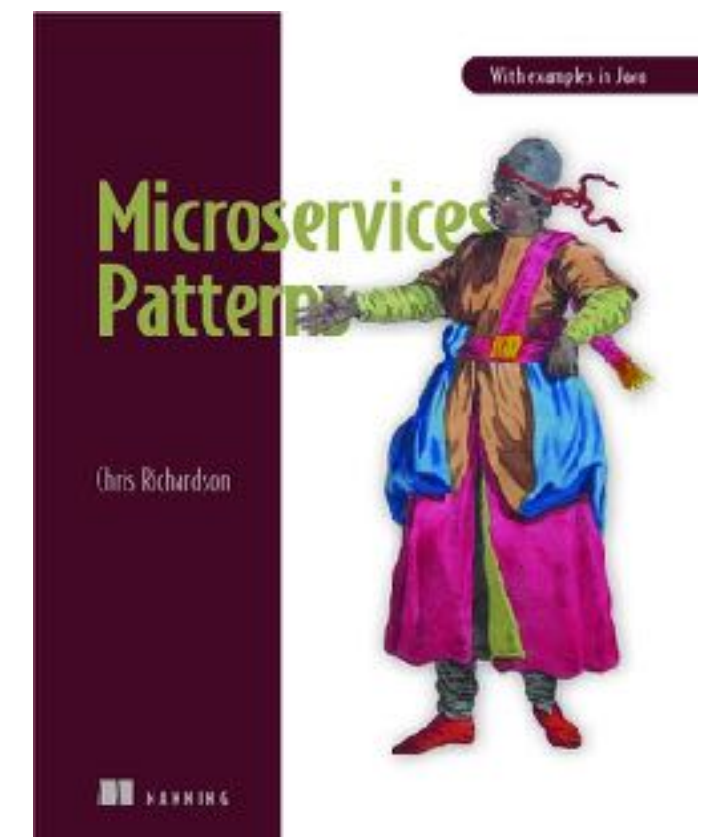


Testing

Reengineering

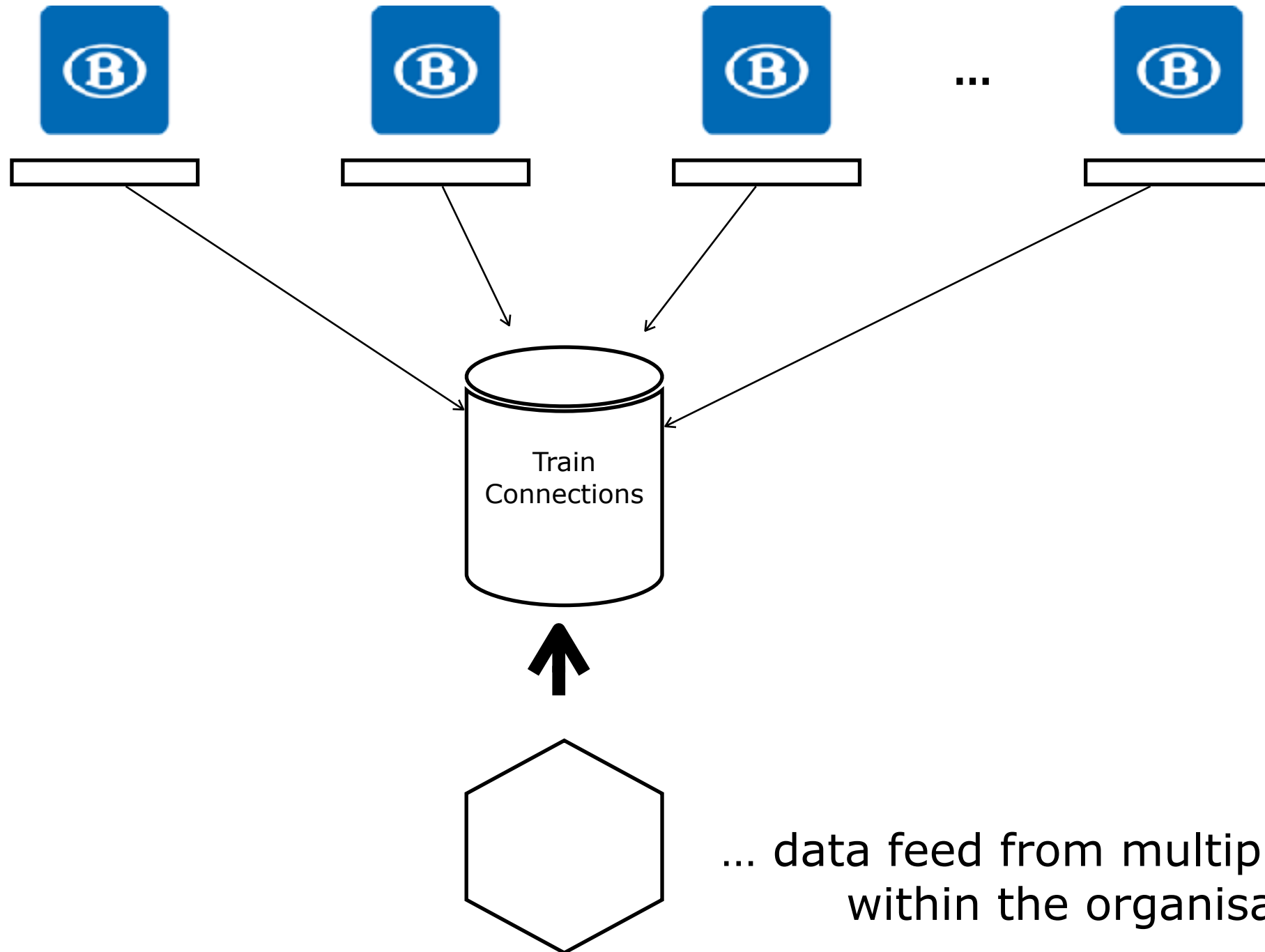


Microservices



Context: (Train) App

Flexible demand with sometimes peak volume in transactions



Pattern: MicroServices

Context

- Distributed system (cloud) with multiple access points
 - + Many read access - few write and update

Problem

- Elastic scaling of access points to deal with peak demand

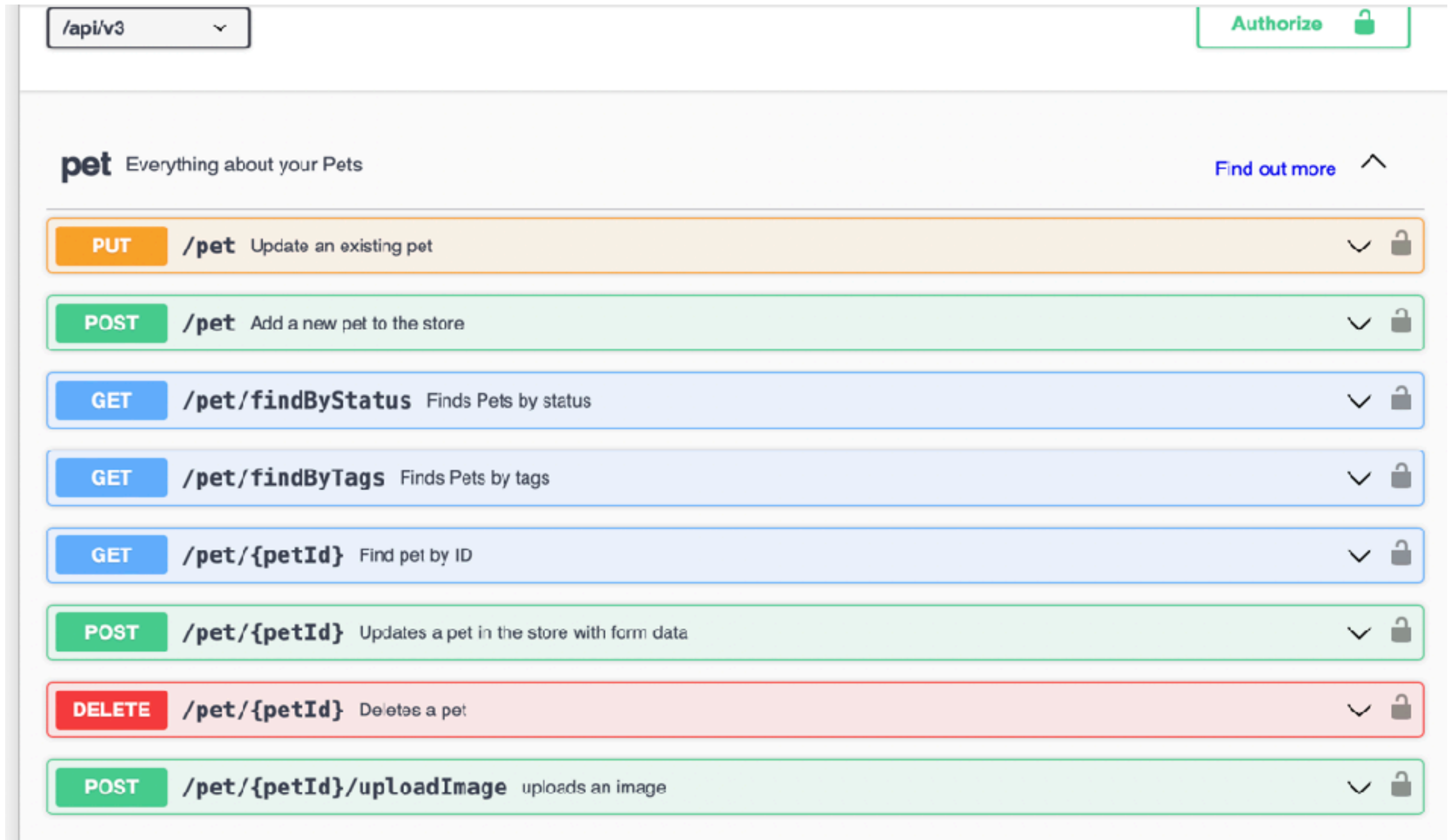
Solution

- *Microservices* structure an application as a collection of small, loosely coupled and independently deployable services.
 - + Each of these services corresponds to a specific business functionality and can be developed, deployed and *scaled* independently.
- Each service is independent and communicates with others via well-defined APIs and protocols (REST-API)

Tradeoffs

- How much data sharing is needed?
 - + Database per service (Database sharding — elastic split of database)
 - + Event mechanism to notify updates
- How much communication needed?
 - Each service deployed by separate DevOps team.
 - Business transactions that span multiple services? (the Saga pattern)
- Resilience: what if a service is down?
 - + Reroute calls to failing service (the Circuit breaker pattern)

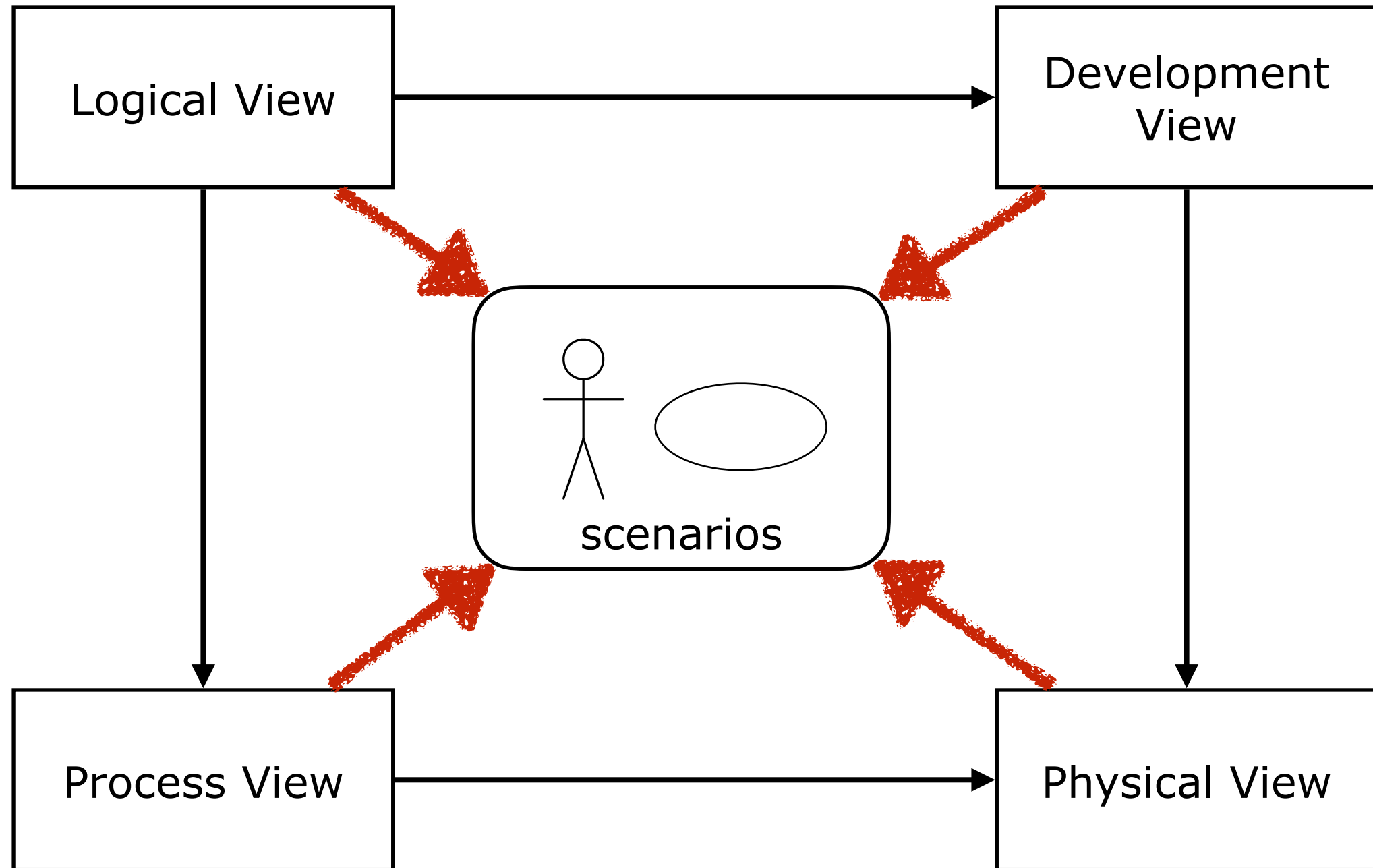
MicroService Example - Pet Store (REST API)



The image shows the Swagger UI for the Pet Store REST API. At the top left, there is a dropdown menu for the API version, currently set to `/api/v3`. At the top right, there is an **Authorize** button with a lock icon. Below the header, the API is titled **pet** with the description "Everything about your Pets" and a **Find out more** link. The main content area lists eight API endpoints, each with a colored button indicating the HTTP method, the endpoint path, a description, and a lock icon indicating authentication requirements.

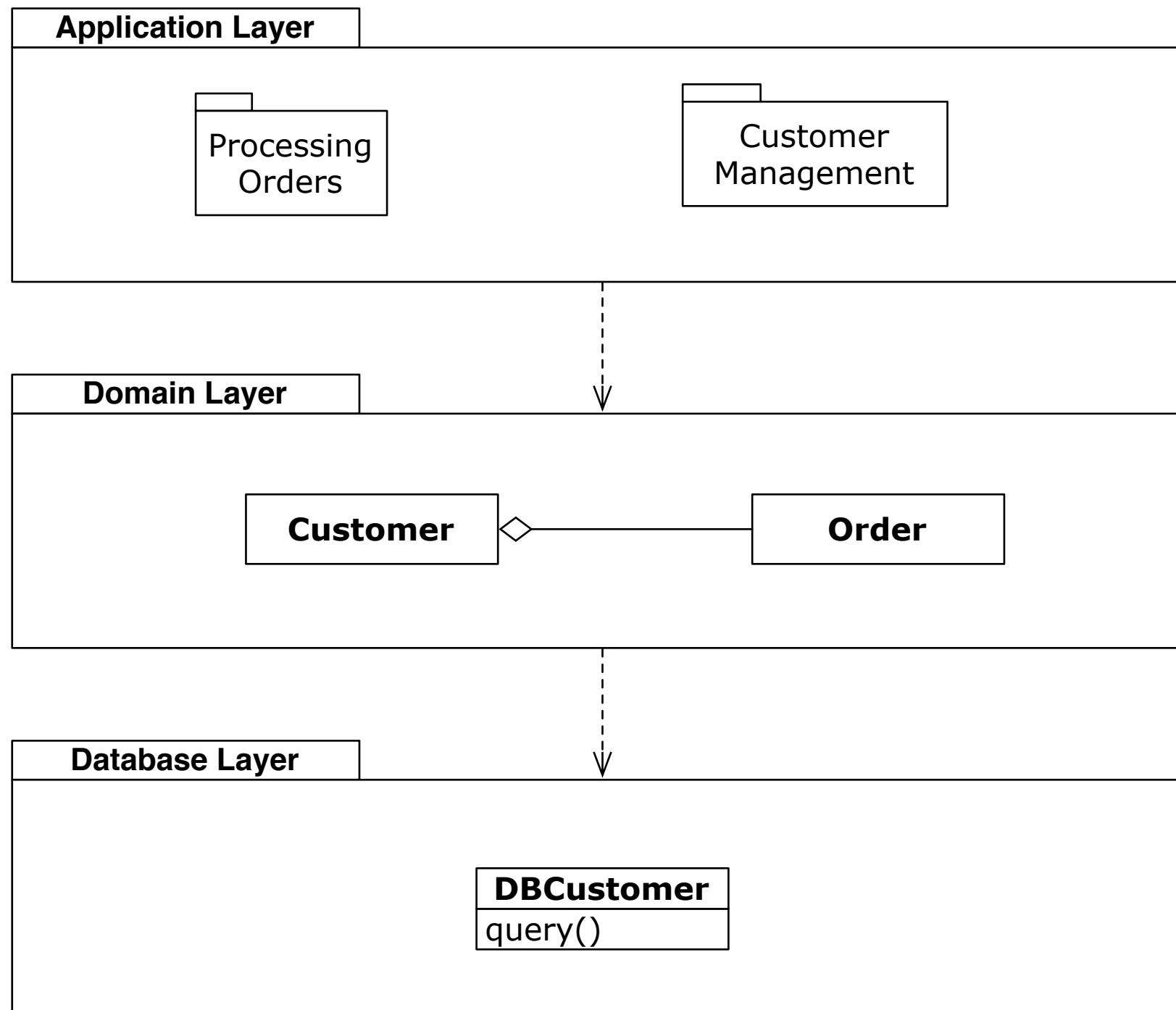
Method	Endpoint	Description	Authenticated
PUT	<code>/pet</code>	Update an existing pet	Yes
POST	<code>/pet</code>	Add a new pet to the store	Yes
GET	<code>/pet/findByStatus</code>	Finds Pets by status	Yes
GET	<code>/pet/findByTags</code>	Finds Pets by tags	Yes
GET	<code>/pet/{petId}</code>	Find pet by ID	Yes
POST	<code>/pet/{petId}</code>	Updates a pet in the store with form data	Yes
DELETE	<code>/pet/{petId}</code>	Deletes a pet	Yes
POST	<code>/pet/{petId}/uploadImage</code>	uploads an image	Yes

4+1 architectural view model



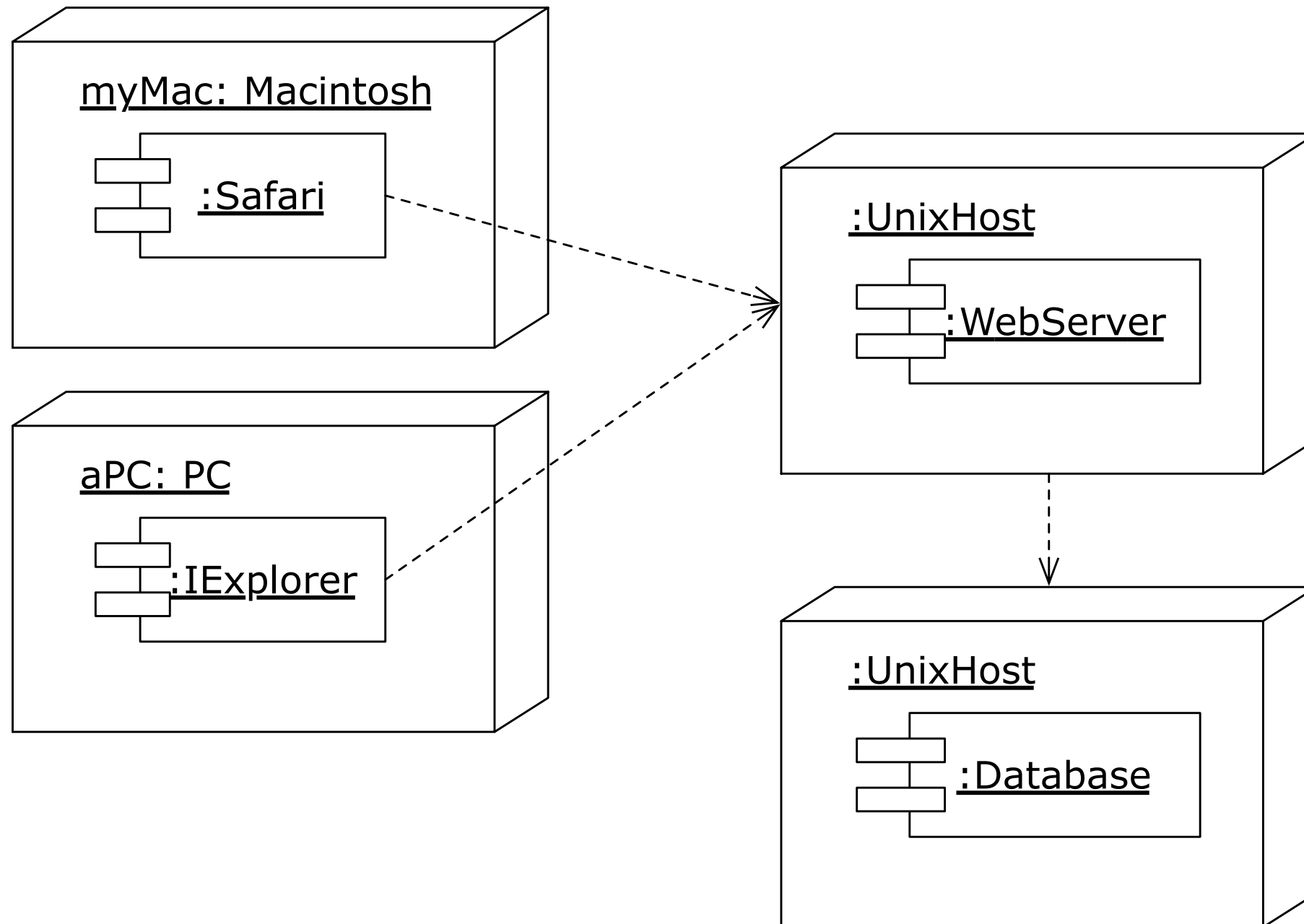
UML: Package Diagram

Decompose system in packages (containing any other UML element, incl. packages)



UML: Deployment Diagram

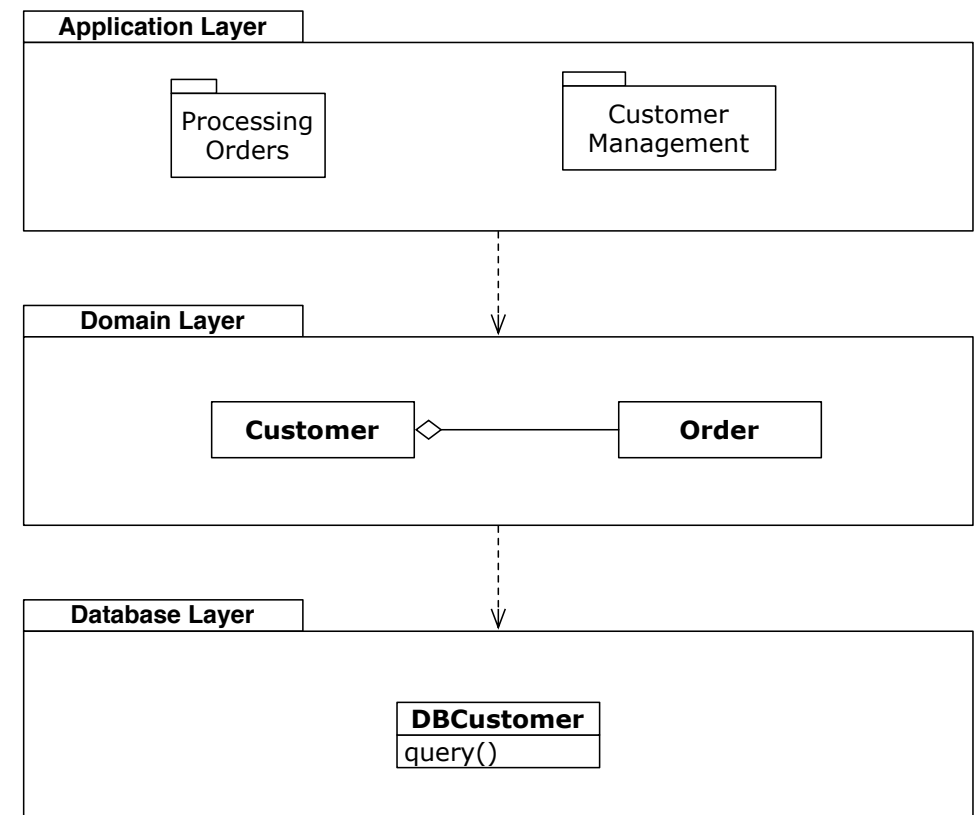
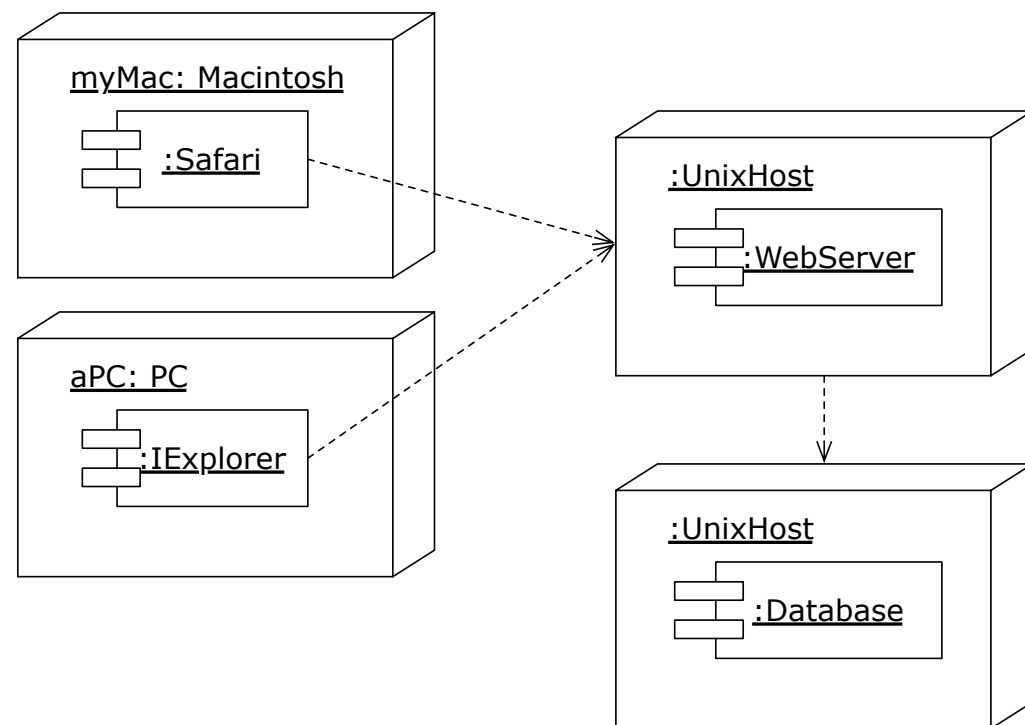
Shows physical lay-out of run-time components on hardware nodes.



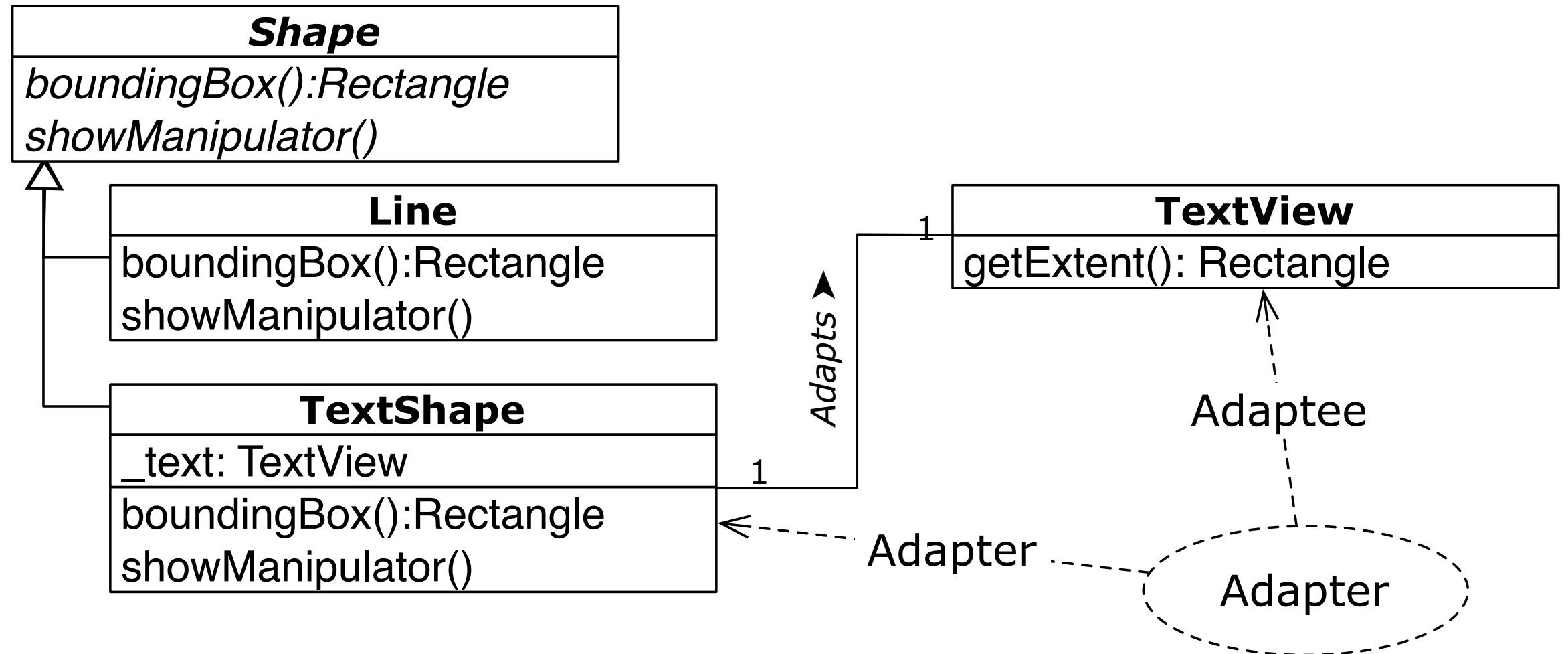
Deployment Diagram vs Package Diagram



- What's the distinction between a package diagram and a deployment diagram?
- Which one would you use in the 4+1 architectural views?
(logical view / development view /
process view / physical view)



UML: Patterns



Architecture Assessment

Why?

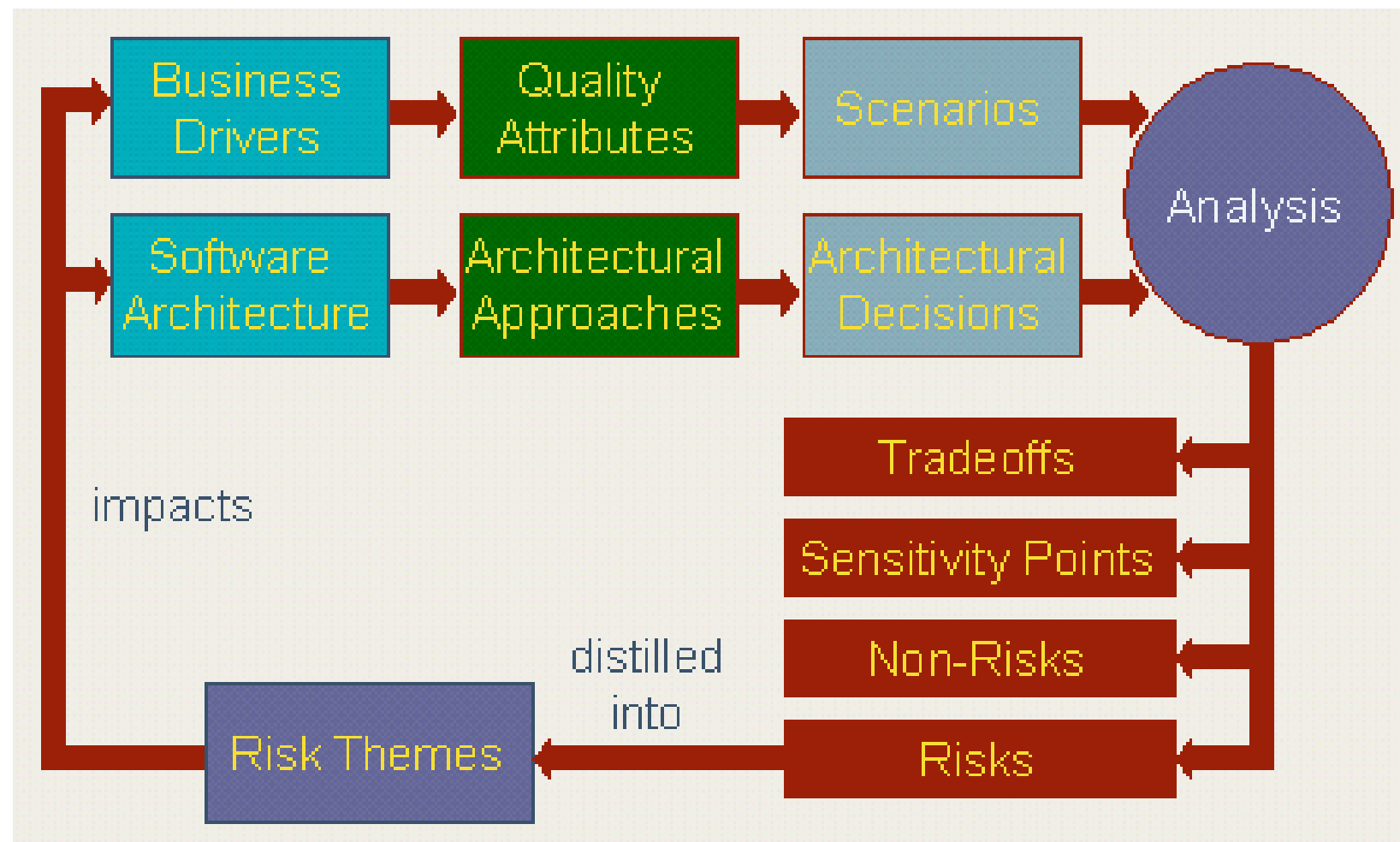
- The earlier you find a problem in a software project, the better.
 - + Identify and assess risks!
- An unsuitable architecture is a recipe for disaster.
 - + A poor architectural design cannot be rescued by good construction technology.
 - + If you wait until the system is built, tackling architectural problems comes at a great cost

Architecture evaluation is a cheap way to avoid disaster.

- Organize review early in the process
 - + An architecture evaluation doesn't tell you "yes" or "no" or "6,75 out of 10".
 - > It tells you where the risks are.

Architecture Tradeoff Analysis Method(ATAM)

- originated from Software Engineering Institute (SEI) at Carnegie Mellon



Answers to two kind of questions:

- Is the architecture *suitable* for the system for which it was designed?
- Which of two or more competing architectures is the most *suitable* one for the system at hand?

ATAM Terminology

Risks are potentially problematic architectural decisions.	The rules for writing business logic modules in the second tier of your three-tier client-server style are not clearly articulated. This could result in replication of functionality, thereby compromising modifiability of the third tier.
Nonrisks are good decisions that rely on assumptions that are frequently implicit in the architecture.	Assuming message arrival rates of once per second, a processing time of less than 30 milliseconds, and the existence of one higher priority process, then a one-second soft deadline seems reasonable.
A sensitivity point is a property of one or more components (and/or component relationships) that is critical for achieving a particular quality attribute response.	The average number of person-days of effort it takes to maintain the system might be sensitive to the degree of encapsulation of its communication protocols and file formats.
A trade-off point involves two (or more) <i>conflicting</i> sensitivity points.	If the processing of a confidential message has a hard real-time latency requirement then the level of encryption could be a trade-off point.

Architecture in scrum?

*** Revised ***

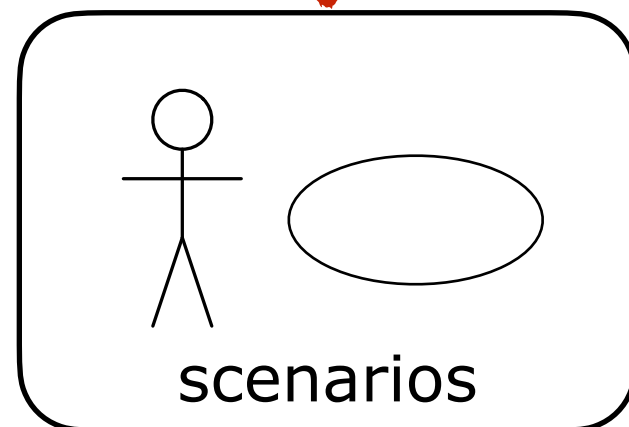
Spike (a.k.a. Knowledge Acquisition Stories / Proof-of-concept)

Spike

As a developer
I want to prototype two alternatives for
the ... component
so that I know

Conditions of Satisfaction

- Run Speed Tests
- Run Load Tests
- Run Security Tests
- Write short memo comparing the results

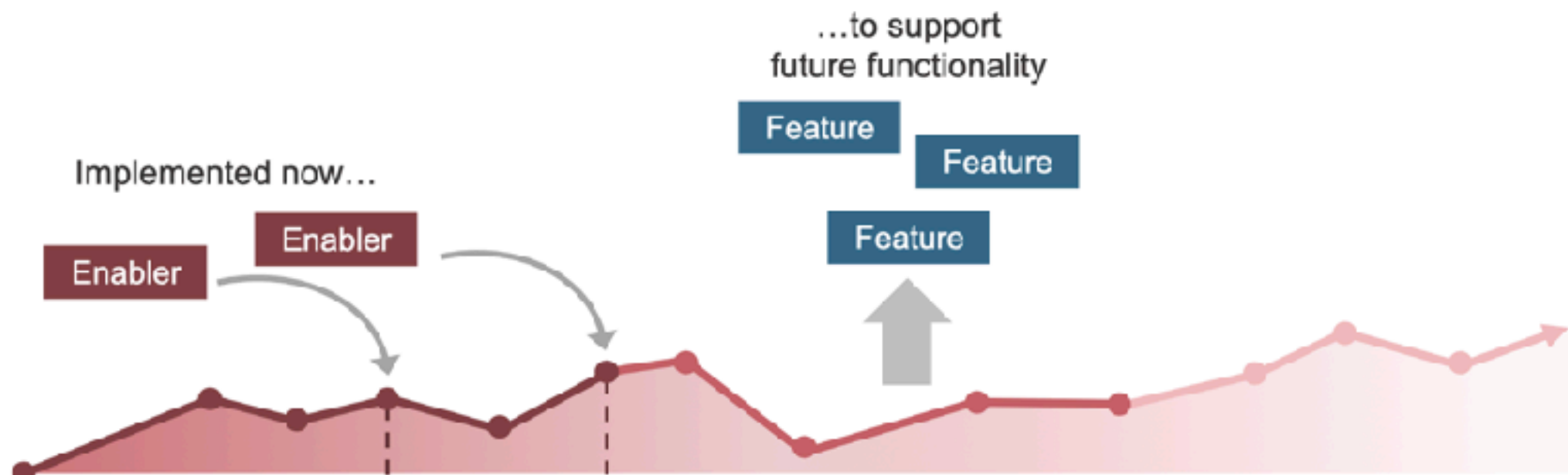


Link the Spike Story to the
user story/use case scenario

*** Revised ***

While we must acknowledge emergence in design and system development, a little planning can avoid much waste. —James Coplien, Lean Architecture

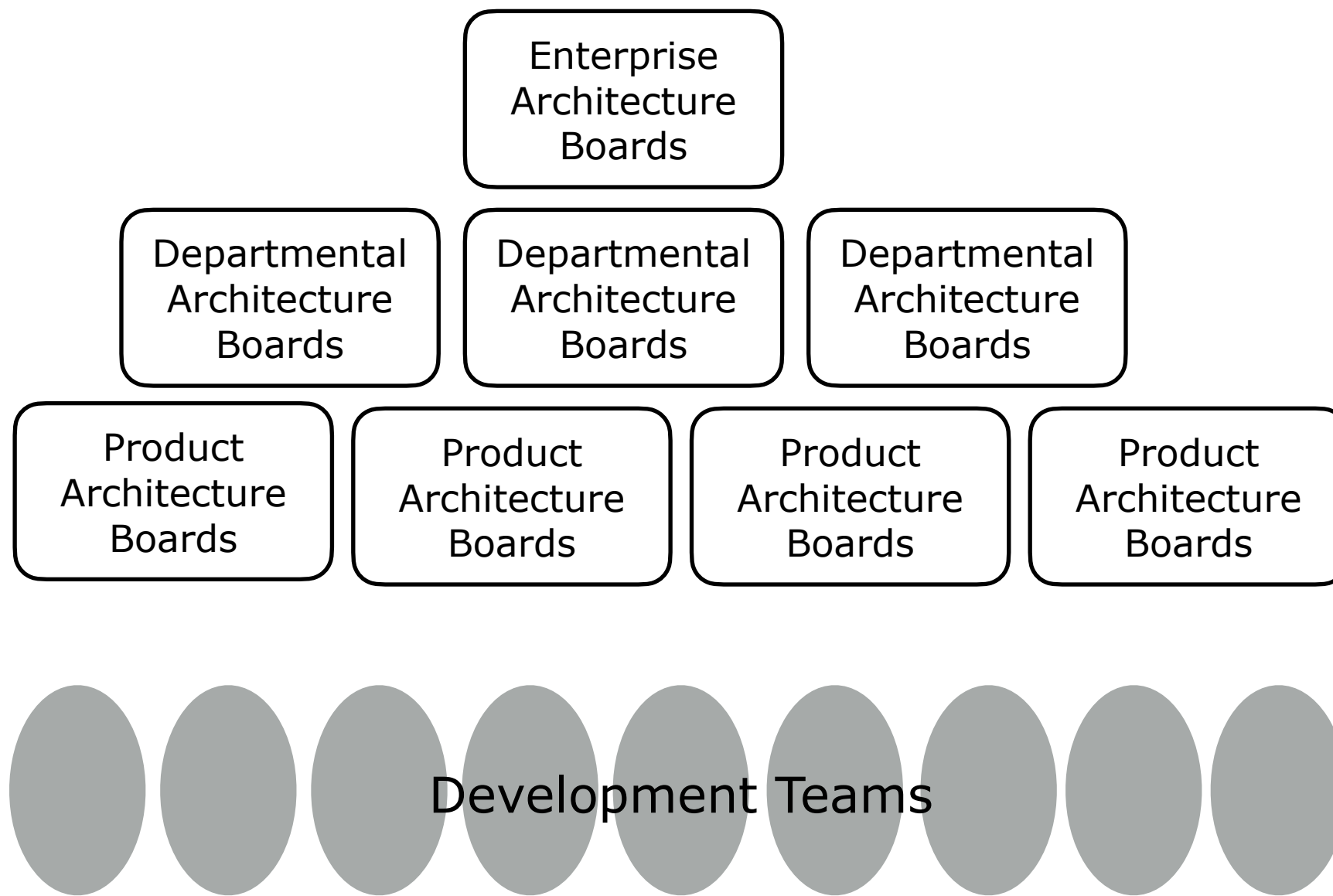
- Agile development avoids big design up-front
 - *emergent design*—defining and extending the architecture only as necessary to deliver the next increment of functionality.
 - *intentional architecture* — requires some centralized planning and cross-team coordination



© Scaled Agile, Inc.

Levels of Architectural Decisions

*** New ***



Guiding Principles

Architectural Decisions

GuardRails

Guiding Principles

*** New ***

A declarative statement made with the intention of guiding architectural design decisions in order to achieve one or more qualities of a system.

[Erd21] Continuous Architecture in Practice

Examples

- Reuse before buy before build
- Leave the codebase cleaner than you found it
(*the boyscout rule*)
- Avoid duplication of functionality
- Given the nature of the business, quality takes precedence of cost over timing

Architectural Decision Record (Y-statement)

*** new ***

See also <https://adr.github.io/adr-templates/>

*In the context of <use case uc
and/or component co>,*

... facing <non-functional concern c>,

.. we decided for <option o1>

and neglected <options o2 to on>,

... to achieve <quality q>,

... accepting downside <consequence c>.

GuardRails

*** Revised ***

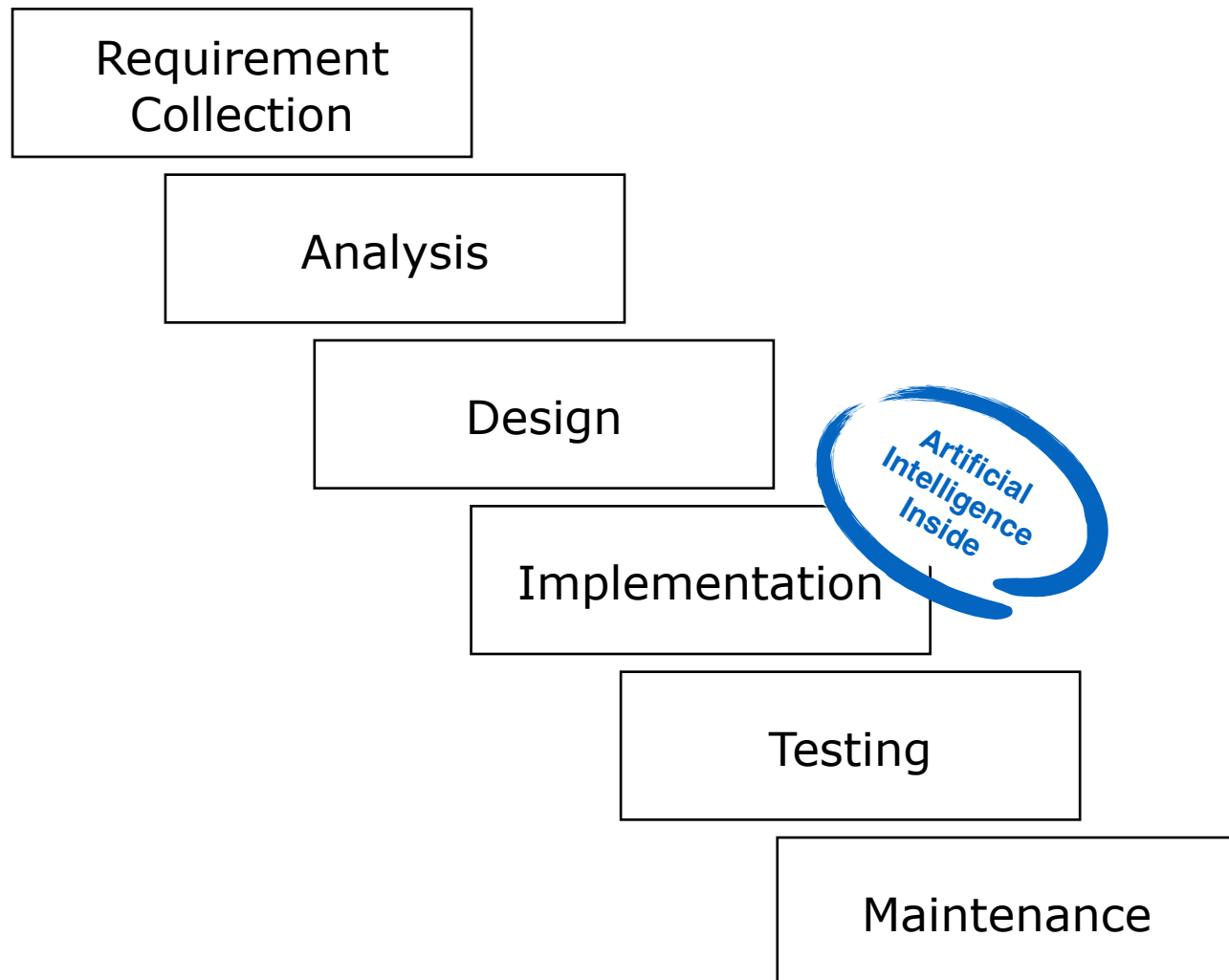
- rules, standards and best practices related to the development pipeline
 - + coding, building, testing, release, *design*, ...
- Enforced by
 - + reviews
 - + tool based CI/CD quality checks



- Staying behind the guardrails = proceed without consulting other teams
- Moving outside = additional discussion or approval needed
 - + Changing existing guardrails?
 - + Adopting new guardrails?

Generative AI & Software Life Cycle

*** new ***

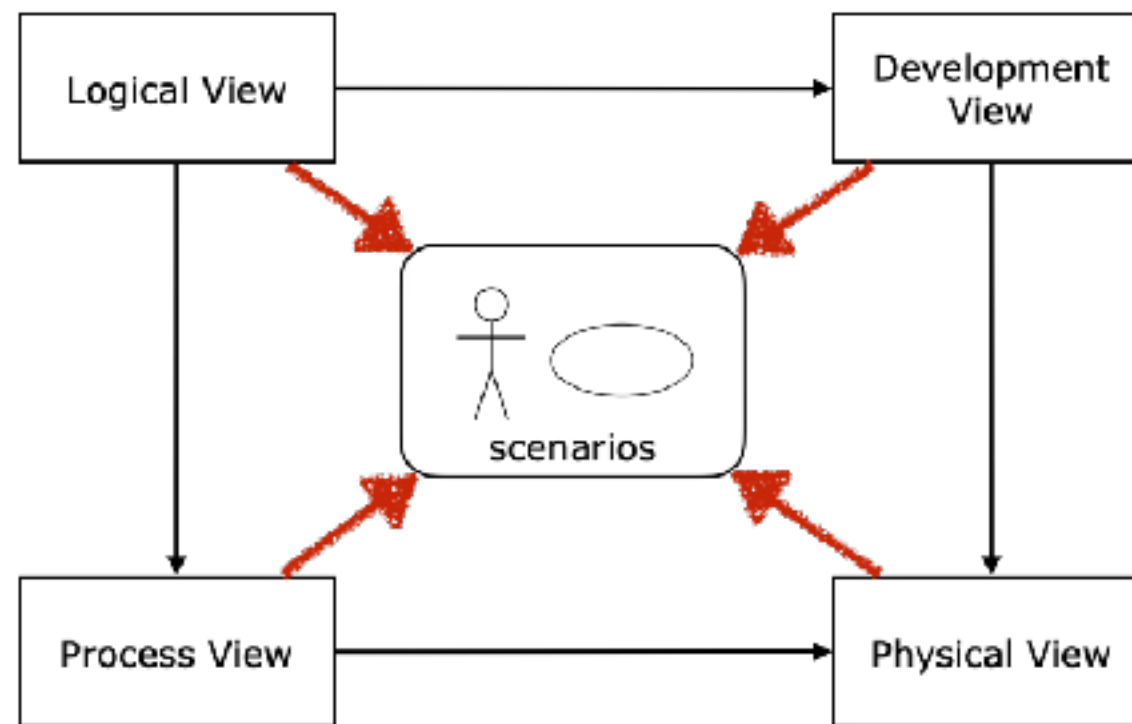


Generative AI makes writing code nearly free.

... everything else becomes *more* important.

Generative AI & Software Architecture

*** new ***



Steer Generative AI by means of
Clear Structure & Enforced Guardrails

Beware

Patterns

- Patterns define the essence of the solution
 - > misinterpretation is common among people
- Patterns are “Expert” knowledge
 - > “hammer looking for a nail” syndrome
- Patterns introduce complexity (more classes, methods, ...)
 - > cost/benefit analysis

Architecture

- Architecture intends to tackle complexity
 - > say less with more
- Architecture implies tradeoffs
 - > a boxes and arrows diagram is not an architecture (at least consider coupling/cohesion)
- *Generative AI demands an Explicit Architecture*
 - > *logical view / development view / process view / physical view*
 - > *enforced guardrails*

Correctness & Traceability

Correctness

- Are we building the system right?
 - + Architecture deals with non functional requirements
 - Choosing the best architecture involves tradeoffs
 - + Architecture allows to scale up
 - Organize (testing) work in the team
- Are we building the right system?
 - + Indifferent



Traceability

- Requirements $\Leftrightarrow$ System?
 - + Architecture implies extra abstraction level
 - + Software architecture is intangible
 - Traceability becomes more difficult
 - *Generative AI can help (challenge assumptions)*



Summary (i)

You should know the answers to these questions

- What's the role of a software architecture?
- What is a component? And what's a connector?
- What is coupling? What is cohesion? What should a good design do with them?
- What is a pattern? Why is it useful for describing architecture?
- Can you name the components in a 3-tiered architecture? And what about the connectors?
- Why is a repository better suited for a compiler than pipes and filters?
- What's the motivation to introduce an abstract factory?
- Can you give two reasons not to introduce an Adapter (Wrapper)?
- What problem does an abstract factory solve?
- List three tradeoffs for the Adapter pattern.
- How do you decide on two architectural alternatives in scrum?
- What's the distinction between a package diagram and a deployment diagram?
- Define a sensitivity point and a tradeoff point from the ATAM terminology.
- *How does generative AI benefit from software architecture?*

You should be able to complete the following tasks

- Take each of the patterns and identify the components and connectors. Then assess the pattern in terms of coupling and cohesion. Compare this assessment with the tradeoffs.
- *Write a Y-statement motivating the choice for a given design pattern.*

Summary (ii)

Can you answer the following questions?

- What do architects mean when they say “architecture maps function onto form”? And what would the inverse “map form into function” mean?
- How does building architecture relate to software architecture? What’s the impact on the corresponding production processes?
- Can you explain the difference between an architecture and a pattern?
- Explain the key steps of the ATAM method?
- What happens when your team goes outside the boundaries of the guardrail?
- How would you organize an architecture assessment in your team?
- *What is the distinction between (i) a guiding principle, (ii) an architectural decision, (iii) a guardrail*